k-Nearest Neighbours

All the models we saw in Chapter 1, either the strictly linear ones defined by

$$\hat{y} = \hat{f}_{\text{Lin}}(\mathbf{x}) = \mathbf{x}^T \boldsymbol{\beta}^\star,$$

or the ones defined in terms of sets of nonlinear functions $\mathbf{h}(\mathbf{x})$ given by

$$\hat{y} = \hat{f}_{\text{Nonlin}}(\mathbf{x}) = \mathbf{h}(\mathbf{x})^T \boldsymbol{\beta}_h^\star$$

are *global* models. Indeed, each of those models is described by the vector of constant parameters $\boldsymbol{\beta}$, and it applies unchanged over the whole domain of the inputs, i.e., it spans the entire data space.

The alternative is to obtain *local* models, which lead to piece-wise descriptions of the data. In other words, we aim not for one model $\hat{f}$ for any value of the input, but for **a model that is different depending on the input $\mathbf{x}^{\text{in}}$**.

Local models have advantages and disadvantages, but they can be extremely powerful tools in data analysis. We will now introduce perhaps the simplest such model, k-Nearest Neighbours, which has wide applications.

1. The k-Nearest Neighbours (kNN) algorithm: continuous variables

Our aim is to construct a *local model*:

$$\hat{y} = \hat{f}_{\mathcal{N}(\mathbf{x}^{\text{in}})}(\mathbf{x}^{\text{in}}),$$

where $\mathbf{x}^{\text{in}}$ represents a new (unseen) data point (i.e. not in the data set used to train the model) for which we want to make a prediction. Note how, instead of applying a model globally to all the input data, the model $\hat{f}_{\mathcal{N}(\mathbf{x}^{\text{in}})}(\cdot)$ is dependent on the input.

In the case of kNN, we construct a set of models $\left\{\hat{f}_{\mathcal{N}(\mathbf{x}^{\text{in}})}\right\}$, i.e., a set of functions that return different subsets of the data depending on the input. The subsets are chosen based on a neighbourhood of the input $\mathbf{x}^{\text{in}}$ as defined by a chosen metric.

Let us consider continuous input variables $\mathbf{x}^{(i)} \in \mathbb{R}^p,\ y \in \mathbb{R},\ i = 1, \ldots, N$, and let us choose a distance metric in the space of inputs (in this case, $\mathbb{R}^p$), let's denote it by:

$$\|\mathbf{x}^{(i)} - \mathbf{x}^{(j)}\|.$$

Typically, we will choose the Euclidean distance as our metric.

The kNN method proceeds as follows: for any input $\mathbf{x}^{\text{in}}$, find the k closest samples in the training data set (with N^{training} data points) to $\mathbf{x}^{\text{in}}$ according to our chosen distance. This

subset of samples in the training set define the neighbourhood $\mathcal{N}(\mathbf{x}^{\text{in}})$. Then construct a prediction function for $\mathbf{x}^{\text{in}}$ based on the samples in $\mathcal{N}(\mathbf{x}^{\text{in}})$. The prediction function is usually very simple, i.e., a simple averaging. Figure 2.1a) shows an outline of these ideas.

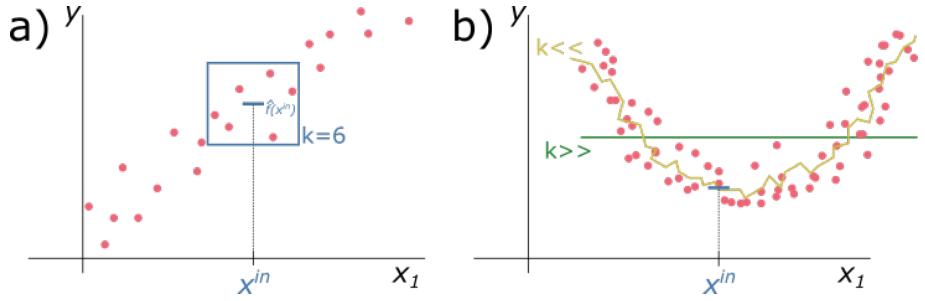


Figure 2.1. k-Nearest Neighbours. a) Given some training data (not necessarily linear like in this cartoon example!), a new data point $\mathbf{x}^{\text{in}}$, and a choice of k (in this case $k = 6$), the algorithm searches for the 6 nearest neighbours to $\mathbf{x}^{\text{in}}$. The prediction for $\mathbf{x}^{\text{in}}$ is then the average of the 6 points' outcome variables, marked by a blue bar. b) The same idea works for more complicated data. But care should be taken when choosing k: overfitting occurs for too small k (yellow line) whereas underfitting occurs for too large k (green line). Hence the parameter k needs to be optimised on the data by a hyperparametric search.

Algorithm: The kNN algorithm is implemented as follows. Given an input $\mathbf{x}^{\text{in}}$:

(1) Compute all distances between $\mathbf{x}^{\text{in}}$ and the samples in the training set:

$$\|\mathbf{x}^{\text{in}} - \mathbf{x}^{(i)}\| \qquad i = 1, \ldots, N^{\text{training}}$$

(2) Find the k nearest neighbours to $\mathbf{x}^{\text{in}}$ which define the neighbourhood:

$$\mathcal{N}_k(\mathbf{x}^{\text{in}})$$

Note that k is a choice, i.e., it is a hyperparameter to set via model selection.

(3) The simplest choice for predictor is just averaging over the values of the outcome variables for the samples in the neighbourhood:

$$\hat{f}_{\mathcal{N}(\mathbf{x}^{\text{in}})} = \frac{1}{k} \sum_{i \in \mathcal{N}_k(\mathbf{x}^{\text{in}})} y^{(i)}$$

The outcome of this algorithm depends on the choices made:

- **Distance metric and standardisation are very important**. Given p predictor variables $\mathbf{x} = (x_1, \ldots, x_p)$ with vastly different ranges, e.g. $1 < x_1 < 10^6$ and $10^{-10} < x_p < 10^{-5}$, it is clear that x_1 will overly dominate the distance measures. Hence, standardising all predictor variables to make them vary on a similar scale is key, unless the hypothesis states that some descriptors indeed are more important than others. By standardisation, we mean replacing every variable $x_m^{(i)}$, $m = 1, ..., p$, $i = 1, ..., N$, by a variable $z_m^{(i)}$ given by:

$$z_m^{(i)} = \frac{x_m^{(i)} - \mu_m}{\sigma_m} \quad \text{with} \quad \mu_m = \frac{1}{N} \sum_{i=1}^{N} x_m^{(i)}, \quad \sigma_m = \sqrt{\frac{1}{N} \sum_{i=1}^{N} (x_m^{(i)} - \mu_m)^2}$$

where μ_m is the mean of the feature m over the sample (so the mean of $z_m^{(i)}$ is zero), and σ_m is its standard deviation over the sample (so the standard deviation of $z_m^{(i)}$ is one).

Similarly, the distance metric can serve to give more or less importance to the different variables.

- **The algorithm works better when the number of input variables, p, is relatively small and the variables are all relevant and not redundant.** In the presence of high-dimensional noisy inputs (i.e., with large p and many descriptors being unrelated to the output) the local prediction can be erratic. Hence, kNN is sensitive to the dimensionality of the data.
- As can be seen in Figure 2.1, the choice of k is crucial. Choosing k does allow us to scan the bias-variance trade-off: from the picture we can see, intuitively, that a small k will be highly accurate for a given input, but very variable across inputs, and thus not very generalisable (hence prone to overfitting), and vice versa. In other words, k acts here equivalently to the parameter λ that modulates the regularisation term in linear regression, and hence the model complexity. How to choose the parameter k is the object of the following section. For an expanded discussion of the bias-variance tradeoff in kNN, see also *Hastie, Tibshirani, Friedman, The Elements of Statistical Learning*, Chap. 2, section 2.3.2 and Chap. 7, section 7.3.1.

2. The general procedure of T-fold cross validation

Remark: Following on from the last point of the previous section, we now ask the question of how to choose k, which is a *hyperparameter*. But this section is much more general—it introduces **a procedure that is applied generally across statistical and machine learning methods, termed T-fold cross validation**. It aims to control for the complexity of the model **to achieve optimal generalisability**. Although this procedure is introduced here using kNN as an illustration, it is used widely for most other models in ML for the hyperparametric search.

To address the problem of the selection of k, we introduce T-fold cross validation. (Note that in the literature it is more often denoted k-fold cross validation, but in order to avoid confusion with our kNN, we use T here.)

The main idea of T-fold cross validation is to split our available data for training (for which we have full knowledge, i.e. predictors and observations) into a training set and a validation set. The model is learnt on the training set and checked against the validation set to see how well it performs. To enhance robustness, this split into training-validation is done T times, so that we reduce the risk of overfitting to the training data.

More specifically, first we split the data $\mathcal{S} = \{\mathbf{x}^{(i)}, y^{(i)}\}$ for $i = 1, \ldots, N^{\text{training}}$ into T equal subsets (or "folds") called $\mathcal{S}_t$, such that:

$$\mathcal{S} = \bigcup_{t=1}^{T} \mathcal{S}_t \text{ and } |\mathcal{S}_t| = \frac{|\mathcal{S}|}{T}, \text{ where } |\mathcal{S}| = N^{\text{training}}$$

Then, we set aside one of the subsets ($\mathcal{S}_t$, the validation subset) and train the model on the rest of the samples, i.e., on the complement set $\overline{\mathcal{S}_t} = \mathcal{S} - \mathcal{S}_t$. We learn a model $\hat{f}_{\overline{\mathcal{S}_t}}$, which we use to predict $\mathcal{S}_t$. We then compute an error measure for this prediction on $\mathcal{S}_t$ (in this case, the mean square error, MSE):

$$\text{MSE}_t = \frac{1}{|\mathcal{S}_t|} \sum_{i \in \mathcal{S}_t} \left[\hat{f}_{\overline{\mathcal{S}_t}}(\mathbf{x}^{(i)}) - y^{(i)} \right]^2$$

The same process is done in turn for each of the subsets of our split $\mathcal{S}_t, t = 1, \ldots, T$ in each case obtaining a different model from their corresponding complement. Finally, we compute the average MSE over all T 'folds' (or splits):

$$\langle \text{MSE} \rangle = \frac{1}{T} \sum_{t=1}^{T} \text{MSE}_t$$

This average MSE can be seen as a measure of how well the model predicts out-of-sample, i.e., on data unseen during the training.

The T-fold cross validation is applied to the data that we decide to use for *training* the method, that is, to learn its parameters having set the hyperparameters. **To test the model after having set the hyperparameters, we need to evaluate it on a portion of the data, of size $N^{\text{test}} = N - N^{\text{training}}$, which is not used in the training nor in the cross-validation procedure**. We call it *test set*, and it needs to be split from the data to use for training before any training or cross-validation step.

Example application to kNN. Coming back to the question of choosing k for the kNN model, this procedure can of course be done for a range of values of k, i.e. computing $\langle \text{MSE}(k) \rangle$. As a result, one might obtain a graph similar to Figure 2.2, from which one can obtain an optimal $k^\star$ (or a range of 'good' k values in more realistic scenarios).

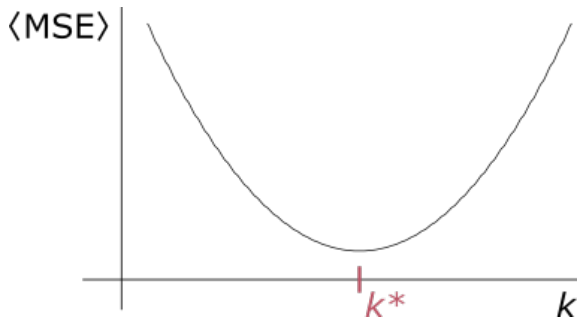


Figure 2.2. A schematic of the outcome of scanning k for a kNN model through T-fold cross-validation. This can help us choose an optimal $k^\star$. For small k, the fold-averaged MSE will be large, since the model will not be generalisable (refer to Figure 2.1) and thus will be performing badly when predicting out-of-sample. Similarly for large k, the model will be too generic and thus also performs badly. Therefore, there will be a k, i.e. a level of complexity of the model, which will be optimal. In practice, such nice behaviour is rarely observed but one can obtain good regions for the relevant parameter. Alternatively, one can find that certain parameters have little or no influence on the out-of-sample prediction, so they can be set to a value that does not increase unnecessarily the model complexity.

How to choose the number of folds T: Whilst cross-validation helps choose a k, what about choosing the number of splits T? It seems clear that the way in which we split the data into training and validation, and the balance between both sets, will affect the results of the cross-validation.

The extreme case is $T = N^{\text{training}}$, which is called *leave-one-out* cross-validation (LOO-CV). This tests how well each sample can be predicted from the rest of samples. However, this is computationally expensive and not necessary.

In practice, **the number of folds T is chosen based on the size of the data set and the computing power available** (larger T means more models need fitting). Commonly, $T = 5$ is used for a smaller data set and $T = 10$ for larger ones, so that one

does out-of-sample prediction on validation sets that contain, respectively, around 20 or 10% of the data.

3. k-Nearest Neighbours for discrete variables

3.1. kNN for discrete predictors. The kNN algorithm can also be applied to discrete data by extending the concepts described above. For example, let us suppose that our data is binary:

$$\mathbf{x}^{(i)} \in \{0,1\}^p, \quad y^{(i)} \in \mathbb{R}$$

Then, in order to use the kNN algorithm, **all we have to do is to define a distance measure on the input set of binary variables**, e.g. the Hamming distance:

$$d_H(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}) = \sum_{m=1}^{p} I(x_m^{(i)} \neq x_m^{(j)})$$

Here we have made use of a version of the *indicator function*, which in this case, can simply be defined as:

$$I(b) = \begin{cases} 1, & \text{if } b \text{ is true} \\ 0, & \text{if } b \text{ is false} \end{cases}$$

In other words, the Hamming distance is the number of positions at which the corresponding entries are different. The rest of the kNN algorithm is exactly the same: this shows that defining a distance measure is key to incorporating any type of predictor spaces.

3.2. k-Nearest Neighbours with discrete outcomes. The previous sections addressed a *regression problem* where the input variables were discrete but the outcomes were continuous variables. We concluded that this can be accommodated easily by changing to a distance metric in the space of discrete inputs.

We now show that kNN can also be applied to classification, i.e. when the outcomes are discrete variables belonging to Q classes:

$$y^{(i)} \in \mathcal{C}_q = \{c_1, \ldots, c_Q\}$$

The inputs can be continuous or discrete, as this will be accommodated through the appropriate distance metric $d(\mathbf{x}^{(i)}, \mathbf{x}^{(j)})$.

The kNN algorithm for discrete outcomes is similar to the continuous version but adapting the *predictor* function. Schematically, the steps are:

(1) Choose k – same as before;

(2) For $\mathbf{x}^{\text{in}}$, at fixed k, find the neighbourhood $\mathcal{N}_k(\mathbf{x}^{\text{in}})$ – same as before;

(3) The final prediction is different here: $\hat{y} = \hat{f}(\mathbf{x}^{\text{in}}) \in \mathcal{C}_Q$ by the majority rule.

This is best understood visually, see Figure 2.3 for a descriptive intuition. The idea is simple: $\mathbf{x}^{\text{in}}$ will be assigned the class of the majority of its neighbours.

The *majority rule* is the simplest predictor function in this case, and can be seen as a discretisation of a probabilistic predictor, as follows.

Recall that $\mathcal{C}_q = \{c_1, \ldots, c_Q\}$ represents the set of all classes and I is the indicator function. Then we define the output predictor in terms of the probability that the input

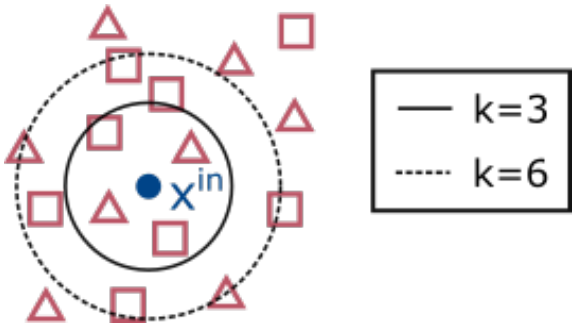


Figure 2.3. k-Nearest Neighbours for a discrete outcome variable. Here we have some data belonging to one of two classes: squares and triangles in some predictor space. For $k = 3$, we apply the majority rule to count two triangles and one square in the neighbourhood and get a prediction $\hat{y}(\mathbf{x}^{\text{in}}) = \triangle$, whereas for $k = 6$, we have $\hat{y}(\mathbf{x}^{\text{in}}) = \square$. Note that for ease of visualisation, the radii were drawn such that only those data points are counted as inside, if their symbols are fully inside the radius.

$\mathbf{x}^{\text{in}}$ belongs to each of the Q classes:

$$\hat{f}_{\text{Prob-kNN}}(\mathbf{x}^{\text{in}}) = \frac{1}{k} \begin{pmatrix} \sum_{i \in \mathcal{N}_k(\mathbf{x}^{\text{in}})} I\big(y^{(i)} \in c_1\big) \\ \vdots \\ \sum_{i \in \mathcal{N}_k(\mathbf{x}^{\text{in}})} I\big(y^{(i)} \in c_Q\big) \end{pmatrix}$$

$$= \frac{1}{k} \begin{pmatrix} \text{number of neighbour points in } c_1 \\ \vdots \\ \text{number of neighbour points in } c_Q \end{pmatrix}$$

Hence each component of $\hat{f}_{\text{Prob-kNN}}(\mathbf{x}^{\text{in}})$ denotes the probability of $\mathbf{x}^{\text{in}}$ belonging to a specific class. Then the majority rule corresponds to the *argmax function* applied across the components of this probabilistic predictor, i.e., choosing the class with the maximum probability among the Q classes:

$$\hat{f}_{\text{MajRule-kNN}}(\mathbf{x}^{\text{in}}) = \operatorname*{argmax}_q \hat{f}_{\text{Prob-kNN}}(\mathbf{x}^{\text{in}})$$

so the class assignment for a certain input data point is 1 for a certain class, and 0 for all the others.

> *Remark.* This discretisation (the argmax step), in which a probabilistic outcome is transformed into a discrete outcome by choosing the maximum probability, is typical in many classification methods, including neural networks (chapter 7) and logistic regression (chapter 3), as we will see during the rest of the course. In general, we talk about *soft* classification when the predicted assignment to classes is probabilistic, *hard* classification when we perform the argmax discretisation.

Finally, note that to evaluate the classification outcome one will need to define an *error function* (or, conversely, a *quality function*) that works for categorical data. (The MSE is obviously not appropriate, but we will see these measures later on, starting with Section 2 of Chapter 3). Once our quality function is defined, the concept of T-fold cross-validation remains the same as above.